КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп’ютерних наук та кібернетики

звіт до проєкту на тему:

«ДОСЛІДЖЕННЯ ВПЛИВУ РОЗМІРУ ВИБІРКИ ТА СПОСОБУ ВІДБОРУ НА ЯКІСТЬ КЛАСИФІКАЦІЇ»

Виконав студент 2 курсу магістратури
Групи ШІ-2
Задорожний Михайло Олександрович

Київ – 2026

ЗМІСТ

Вступ 2
Опис датасету та постановка задачі 3
Методологія експерименту 3
Методи відбору підвибірок 4
Використані моделі 5
Результати експериментів 5
Висновки 8
Список використаних джерел 8
Додаток А: таблиця середніх та F1 score та 95% довірчих інтервалів для моделі KNN 9
Додаток Б: таблиця середніх та F1 score та 95% довірчих інтервалів для моделі KNN 13
Додаток В: таблиця середніх та F1 score та 95% довірчих інтервалів для моделі Random Forest 16
Додаток Г: код програми 19

Вступ

У задачах машинного навчання питання достатнього обсягу даних є одним із центральних. З одного боку, недостатній розмір вибірки призводить до високої дисперсії оцінок моделі, нестабільних результатів та поганої узагальнювальної здатності. З іншого боку, збір надлишкових даних пов’язаний із часовими, фінансовими та обчислювальними витратами. Тому практично важливо визначити, який мінімальний розмір навчальної вибірки дозволяє досягти прийнятного рівня якості.

У даній роботі досліджується залежність якості класифікації від розміру навчальної вибірки для кількох моделей машинного навчання. Особливий акцент зроблено на впливі способу формування підвибірки. Було реалізовано чотири різні методи відбору даних, після чого для кожного з них побудовано

криві навчання, які відображають зміну F1-score залежно від розміру підвибірки. Для оцінювання стабільності результатів додатково обчислювались 95% довірчі інтервали.

Цільовим критерієм якості було обрано macro F1-score із пороговим значенням 0.90. Окремо аналізувалося, при якому мінімальному розмірі вибірки кожна модель досягає цього рівня якості залежно від способу семплінгу.

Опис датасету та постановка задачі

У роботі використано датасет Palmer Penguins Dataset for EDA[1], який містить морфометричні характеристики пінгвінів трьох різних видів. Кожен запис відповідає окремій особині та містить як числові, так і категоріальні ознаки. До числових характеристик належать довжина дзьоба, глибина дзьоба, довжина ласта та маса тіла. До категоріальних — острів проживання та стать. Цільовою змінною є вид пінгвіна (species).

Перед початком експериментів було виконано базове очищення даних: видалено записи з пропущеними значеннями, категоріальні ознаки приведено до відповідного типу. Після цього дані було розділено на тренувальну та тестову вибірки у пропорції 75% до 25% із збереженням розподілу класів (stratified split). Тестова вибірка залишалась фіксованою протягом усіх експериментів, що забезпечує коректність порівняння результатів між різними підвибірками.

Задача формулюється як багатокласова класифікація. Оскільки кількість об’єктів у різних класах може відрізнятися, для оцінювання якості було обрано macro F1-score. Ця метрика обчислює F1 для кожного класу окремо, після чого усереднює значення, надаючи кожному класу однакову вагу. Таким чином, вона є більш інформативною, ніж accuracy, у випадках можливого дисбалансу.

Методологія експерименту

Експериментальна процедура побудована таким чином, щоб максимально об’єктивно оцінити вплив розміру вибірки на якість моделей. Для цього було згенеровано 40 різних значень розміру підвибірки — точки

були обрані за рівномірим розподілом від мінімально допустимого до повного розміру тренувальної вибірки. Мінімальний розмір визначався як максимум між 10 об’єктами та трикратною кількістю класів, що гарантує хоча б базове представництво кожного класу.

Для кожного значення розміру вибірки виконувалося 100 незалежних повторів семплінгу. На кожній ітерації формувалася нова підвибірка, на якій навчалася модель, після чого її якість оцінювалася на фіксованій тестовій множині. Таким чином, для кожного розміру вибірки формувався розподіл із 100 значень F1-score.

На основі отриманих значень обчислювалися середній F1 та 95% довірчий інтервал за t-розподілом. Це дозволяє оцінити не лише середню якість, а й варіативність результатів, що є критично важливим при роботі з малими вибірками.

Усі експерименти виконувалися з фіксованим random_state для забезпечення відтворюваності результатів.

Методи відбору підвибірок

У межах роботи було реалізовано чотири різні методи семплінгу.

Перший метод — простий випадковий відбір. У цьому випадку з тренувальної вибірки випадковим чином без повернення обирається n об’єктів. Це класичний статистичний підхід, який забезпечує рівну ймовірність потрапляння кожного об’єкта до підвибірки. Проте при малих значеннях n можливе нерівномірне представництво класів.

Другий метод — відбір Бернуллі. Кожен об’єкт включається до підвибірки з імовірністю p, де p дорівнює відношенню бажаного розміру до загального обсягу тренувальних даних. Особливістю цього підходу є те, що фактичний розмір вибірки є випадковим. Це створює додаткову варіативність, особливо для малих p.

Третій метод — систематичний відбір. Визначається крок відбору, після чого обирається випадкова стартова позиція, і далі кожен k-й елемент потрапляє до підвибірки. Такий підхід забезпечує більш рівномірне покриття

простору даних, проте може бути чутливим до прихованої періодичності в порядку записів.

Четвертий метод — стратифікований відбір. У даній роботі страти формувалися за комбінацією острова проживання та виду пінгвіна. Для кожної страти обиралася кількість об’єктів пропорційно її частці в повній вибірці. Такий підхід дозволяє зберігати структуру даних навіть при малих розмірах підвибірки, що зазвичай зменшує дисперсію оцінок.

Використані моделі

Було досліджено три моделі класифікації: Logistic Regression, K-Nearest Neighbors та Random Forest.

Logistic Regression є лінійною моделлю, яка добре працює за умови лінійної роздільності класів. У роботі використовувалась версія з балансуванням класів та збільшеною кількістю ітерацій оптимізації.

K-Nearest Neighbors є непараметричним методом, що класифікує об’єкти на основі найближчих сусідів у просторі ознак. Ця модель суттєво залежить від щільності даних і тому особливо чутлива до зменшення розміру вибірки.

Random Forest є ансамблевим методом, який будує сукупність рішень дерев та агрегує їхні прогнози. Завдяки цьому модель добре узагальнює та є стійкою до шуму.

Перед навчанням числові ознаки масштабувалися за допомогою StandardScaler, а категоріальні кодувалися через OneHotEncoder.

Результати експериментів

Для кожного способу семплінгу було побудовано окремі графіки залежності F1-score від розміру вибірки для трьох моделей.

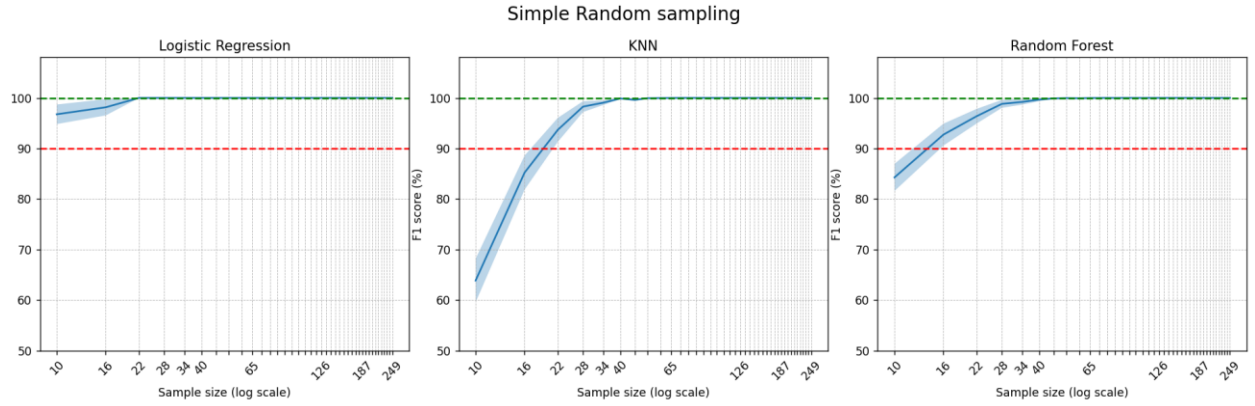

Рис. 1: Візуалізація роботи моделей Logistic Regression (balanced classes), K Nearest Neighbours, Random Forest на простому випадковому відборі– залежність точності від розміру підвибірки. Довірчі інтервали зображені тінями.

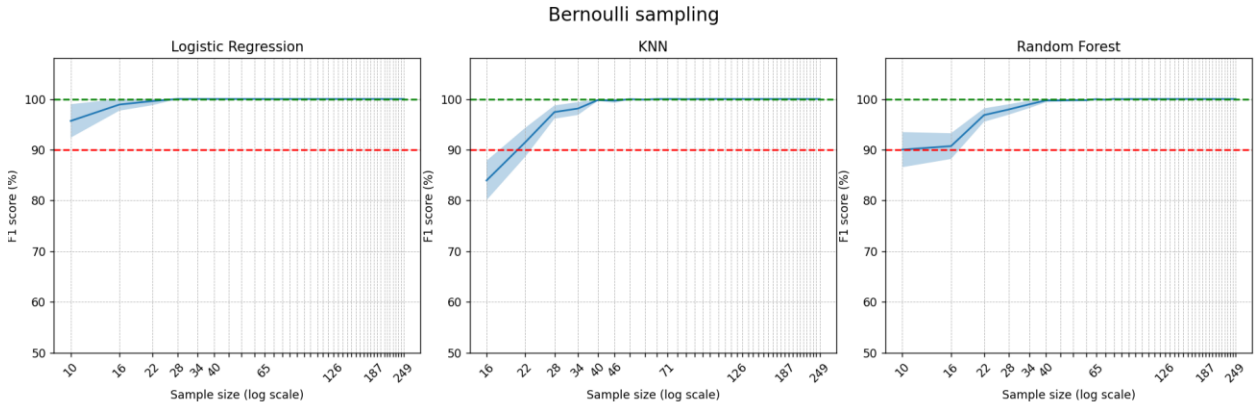

Рис. 2: Візуалізація роботи моделей Logistic Regression (balanced classes), K Nearest Neighbours, Random Forest на відборі Бернуллі – залежність точності від розміру підвибірки. Довірчі інтервали зображені тінями.

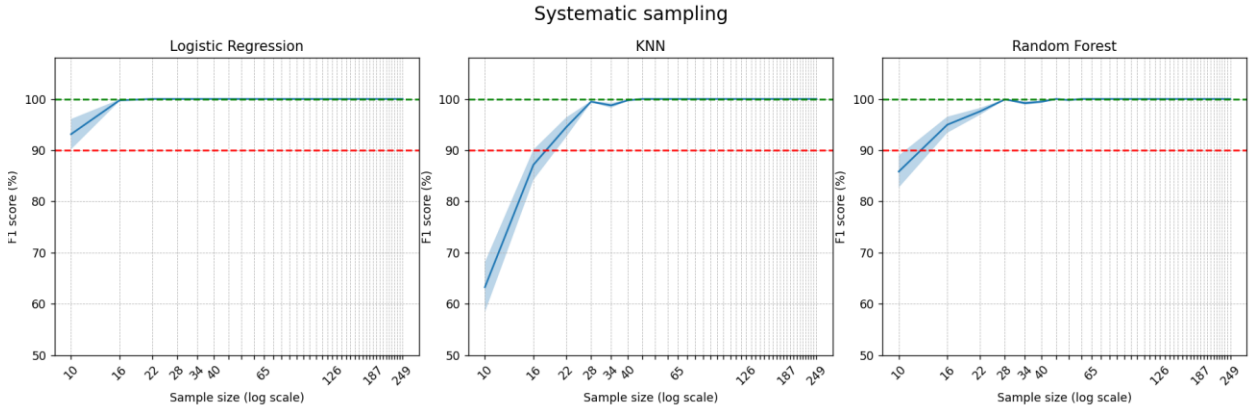

Рис. 3: Візуалізація роботи моделей Logistic Regression (balanced classes), K Nearest Neighbours, Random Forest на систематичному відборі– залежність точності від розміру підвибірки. Довірчі інтервали зображені тінями.

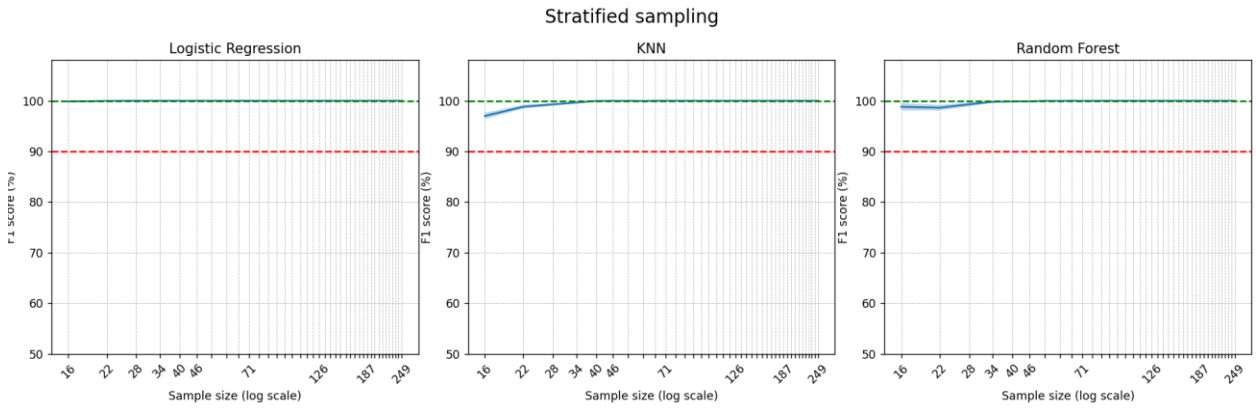

Рис. 4: Візуалізація роботи моделей Logistic Regression (balanced classes), K Nearest Neighbours, Random Forest на стратифікованому відборі– залежність точності від розміру підвибірки. Довірчі інтервали зображені тінями.

Аналіз показує, що в усіх випадках спостерігається монотонне зростання F1-score зі збільшенням розміру вибірки. Проте швидкість цього зростання та ширина довірчих інтервалів суттєво відрізняються залежно від методу відбору:

- Стратифікований відбір демонструє найвужчі довірчі інтервали, що свідчить про високу стабільність результатів. Простий випадковий та систематичний відбір показують схожу поведінку, але меншу стабільність. А відбір Бернуллі характеризується найбільшою варіативністю.
- Простий випадковий та систематичний відбори, за результатами експериментів, здатні найсильнішим чином спотворювати структуру датасету на малих розмірах підвибірок, про що свідчать початкові старотові точки середніх значень F1. Відбір Бернуллі трохи краще з цим справляється, а стратифікований відбір виступає повним переможцем.

Окремо для кожної моделі було сформовано таблиці, у яких для кожного розміру вибірки та способу семплінгу зазначено середній F1 та відповідний довірчий інтервал.

Таблиці для моделей Logistic Regression, KNN та Random Forest прикріплені в додатках A-B відповідно.

Ці таблиці дозволяють визначити мінімальний розмір вибірки, при якому середній F1 перевищує 0.90. Було встановлено, що на датасеті Palmer Penguins Logistic Regression (з балансуванням класів) майже ідеально виконує задачу

буквально з мінімальної кількості, переганяючи навіть Random Forest, хоча різниця невелика, а особливо при стратифікованому відборі. KNN потребує найбільшої кількості даних для стабільної роботи.

Висновки

Проведене дослідження показало, що спосіб формування навчальної вибірки істотно впливає як на середню якість моделі, так і на стабільність результатів. Стратифікований відбір забезпечує найменшу дисперсію та дозволяє досягати високих значень F1 при менших обсягах даних. Простий випадковий та систематичний методи демонструють схожі, але менш стабільні результати. Відбір Бернуллі є найменш стабільним через додаткову випадковість розміру підвибірки.

Серед моделей найкращу стійкість до зменшення вибірки датасету Palmer Pinguins показала логістична регресія, що свідчить про чітку роздільність датасету. модель Random Forest продемонструвала стабільну, але більш поступову динаміку зростання. KNN виявилася найбільш чутливою до розміру навчальної множини.

Таким чином, для задачі класифікації пінгвінів досягнення $F1 \geq 0.90$ можливе навіть при використанні підвибірок значно меншого розміру, ніж повний датасет, за умови застосування стратифікованого семплінгу та достатньо потужної моделі.

Отримані результати та напрацювання за невеликих редагувань можуть бути застосовані до інших датасетів та, відповідно, мають практичне значення для оцінювання мінімально достатнього обсягу даних у прикладних задачах машинного навчання.

Список використаних джерел

1. Rai, S. (n.d.). *Palmer Penguins Dataset for EDA* [Data set]. Kaggle. https://www.kaggle.com/datasets/satyajeetrai/palmer-penguins-dataset-for-eda

2. Kozyrkov, C. (2022, August 17). *Simple random sampling: Is it actually simple? Medium*. https://medium.com/data-science/simple-random-sampling-is-it-actually-simple-71014e58e0d1
3. Santra, R. (2022, October 26). *Sampling techniques. Medium*. https://medium.com/@ritusantra/sampling-techniques-abd1079dc6f6
4. Anello, E. (2021, November 19). *An introduction to probability sampling methods. Medium*. https://medium.com/data-science/an-introduction-to-probability-sampling-methods-7a936e486b5
5. Menon, S. (2020, December 6). *Stratified sampling in machine learning. Medium*. https://medium.com/analytics-vidhya/stratified-sampling-in-machine-learning-f5112b5b9cfe
6. Kant, S. (2022, February 4). *Two stage stratified random sampling — clearly explained. Medium*. https://medium.com/data-science/two-stage-stratified-random-sampling-clearly-explained-59788b110a84
7. Shulga, D. (2018, March 20). *A (very) friendly introduction to confidence intervals. Medium*. https://medium.com/data-science/a-very-friendly-introduction-to-confidence-intervals-9add126e714

Додаток А: таблиця середніх та F1 score та 95% довірчих інтервалів для моделі KNN

Умовні позначення:

- Reached – середнє значення F1 серед повторень для заданої комбінації (тип відбору, розмір підвибірки) перевищує задане порогове значення у 90% F1.
- Not reached – середнє значення F1 менше порогового;
- Declined – в процесі запису повторень надто мало підвибірок були більші мінімального розміру вибірки.

sample_size	Simple Random	Bernoulli	Systematic	Stratified
10	REACHED. F1 mean: 96.8. F1 CI [94.8, 98.7]	REACHED. F1 mean: 95.7. F1 CI [92.4, 99.0]	REACHED. F1 mean: 93.1. F1 CI [90.2, 96.1]	DECLINED: too few valid samples obtained.
16	REACHED. F1 mean: 98.1. F1 CI [96.5, 99.8]	REACHED. F1 mean: 98.9. F1 CI [97.7, 100.1]	REACHED. F1 mean: 99.7. F1 CI [99.6, 99.9]	REACHED. F1 mean: 99.9. F1 CI [99.7, 100.0]
22	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 99.6. F1 CI [98.8, 100.4]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]
28	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
34	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
40	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
46	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
52	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
59	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
65	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
71	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

77	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
83	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
89	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
95	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
101	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
108	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
114	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
120	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
126	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
132	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
138	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
144	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
150	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
157	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

163	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
169	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
175	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
181	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
187	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
193	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
199	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
206	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
212	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
218	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
224	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
230	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
236	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
242	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

249	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

Додаток Б: таблиця середніх та F1 score та 95% довірчих інтервалів для моделі KNN

Умовні позначення:

- Reached – середнє значення F1 серед повторень для заданої комбінації (тип відбору, розмір підвибірки) перевищує задане порогове значення у 90% F1.
- Not reached – середнє значення F1 менше порогового;
- Declined – в процесі запису повторень надто мало підвибірок були більші мінімального розміру вибірки.

sample_size	Simple Random	Bernoulli	Systematic	Stratified
10	NOT REACHED. F1 mean: 63.9. F1 CI [59.6, 68.2]	NOT REACHED. F1 mean: 63.3. F1 CI [58.4, 68.2]	DECLINED: too few valid samples obtained.	DECLINED: too few valid samples obtained.
16	NOT REACHED. F1 mean: 85.2. F1 CI [81.9, 88.6]	NOT REACHED. F1 mean: 87.1. F1 CI [84.2, 90.1]	NOT REACHED. F1 mean: 84.0. F1 CI [80.1, 87.8]	REACHED. F1 mean: 97.0. F1 CI [96.4, 97.6]
22	REACHED. F1 mean: 93.7. F1 CI [91.3, 96.1]	REACHED. F1 mean: 94.5. F1 CI [92.6, 96.4]	REACHED. F1 mean: 91.5. F1 CI [88.6, 94.3]	REACHED. F1 mean: 98.8. F1 CI [98.4, 99.2]
28	REACHED. F1 mean: 98.3. F1 CI [97.2, 99.3]	REACHED. F1 mean: 99.5. F1 CI [99.3, 99.6]	REACHED. F1 mean: 97.4. F1 CI [96.1, 98.7]	REACHED. F1 mean: 99.3. F1 CI [99.0, 99.6]
34	REACHED. F1 mean: 99.1. F1 CI [98.6, 99.5]	REACHED. F1 mean: 98.7. F1 CI [98.3, 99.2]	REACHED. F1 mean: 98.1. F1 CI [96.9, 99.4]	REACHED. F1 mean: 99.6. F1 CI [99.5, 99.8]
40	REACHED. F1 mean: 99.9. F1 CI [99.8, 100.0]	REACHED. F1 mean: 99.7. F1 CI [99.6, 99.8]	REACHED. F1 mean: 99.8. F1 CI [99.6, 100.0]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]

46	REACHED. F1 mean: 99.6. F1 CI [99.4, 99.9]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 99.7. F1 CI [99.3, 100.1]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
52	REACHED. F1 mean: 99.9. F1 CI [99.9, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
59	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 99.9. F1 CI [99.8, 100.0]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]
65	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
71	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
77	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
83	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
89	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
95	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
101	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
108	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
114	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
120	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
126	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

132	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
138	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
144	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
150	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
157	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
163	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
169	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
175	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
181	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
187	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
193	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
199	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
206	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
212	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

218	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
224	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
230	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
236	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
242	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
249	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

Додаток В: таблиця середніх та F1 score та 95% довірчих інтервалів для моделі Random Forest

Умовні позначення:

- Reached – середнє значення F1 серед повторень для заданої комбінації (тип відбору, розмір підвибірки) перевищує задане порогове значення у 90% F1.
- Not reached – середнє значення F1 менше порогового;
- Declined – в процесі запису повторень надто мало підвибірок були більші мінімального розміру вибірки.

sample_size	Simple Random	Bernoulli	Systematic	Stratified
10	NOT REACHED. F1 mean: 84.3. F1 CI [81.6, 86.9]	REACHED. F1 mean: 90.0. F1 CI [86.6, 93.5]	NOT REACHED. F1 mean: 85.8. F1 CI [82.7, 89.0]	DECLINED: too few valid samples obtained.
16	REACHED. F1 mean: 92.7. F1 CI [90.6, 94.9]	REACHED. F1 mean: 90.7. F1 CI [88.2, 93.3]	REACHED. F1 mean: 95.0. F1 CI [93.4, 96.5]	REACHED. F1 mean: 98.8. F1 CI [98.1, 99.5]

22	REACHED. F1 mean: 96.4. F1 CI [94.9, 97.8]	REACHED. F1 mean: 96.8. F1 CI [95.5, 98.2]	REACHED. F1 mean: 97.6. F1 CI [96.9, 98.2]	REACHED. F1 mean: 98.6. F1 CI [98.1, 99.2]
28	REACHED. F1 mean: 98.8. F1 CI [98.0, 99.6]	REACHED. F1 mean: 98.0. F1 CI [96.9, 99.0]	REACHED. F1 mean: 99.9. F1 CI [99.9, 100.0]	REACHED. F1 mean: 99.3. F1 CI [98.9, 99.7]
34	REACHED. F1 mean: 99.2. F1 CI [98.7, 99.7]	REACHED. F1 mean: 98.9. F1 CI [98.2, 99.7]	REACHED. F1 mean: 99.2. F1 CI [98.9, 99.5]	REACHED. F1 mean: 99.8. F1 CI [99.7, 99.9]
40	REACHED. F1 mean: 99.7. F1 CI [99.4, 100.0]	REACHED. F1 mean: 99.7. F1 CI [99.4, 100.1]	REACHED. F1 mean: 99.5. F1 CI [99.2, 99.7]	REACHED. F1 mean: 99.8. F1 CI [99.7, 100.0]
46	REACHED. F1 mean: 99.9. F1 CI [99.8, 100.0]	REACHED. F1 mean: 99.7. F1 CI [99.5, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 99.9. F1 CI [99.8, 100.0]
52	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]	REACHED. F1 mean: 99.8. F1 CI [99.5, 100.1]	REACHED. F1 mean: 99.8. F1 CI [99.7, 99.9]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]
59	REACHED. F1 mean: 99.9. F1 CI [99.9, 100.0]	REACHED. F1 mean: 99.8. F1 CI [99.6, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]
65	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
71	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 99.9. F1 CI [99.7, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
77	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
83	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [99.9, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
89	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
95	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
101	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

108	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
114	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
120	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
126	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
132	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
138	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
144	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
150	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
157	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
163	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
169	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
175	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
181	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
187	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

193	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
199	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
206	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
212	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
218	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
224	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
230	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
236	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
242	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]
249	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]	REACHED. F1 mean: 100.0. F1 CI [100.0, 100.0]

Додаток Г: код програми

```
import os

import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import f1_score
from scipy.stats import t

import datetime
from tqdm import tqdm
from joblib import Parallel, delayed
# ==========================
# Global Hyperparameters
# ==========================

RANDOM_STATE = 42          # Reproducibility
N_REPEATS = 100              # Repeated samplings per sample size
N_POINTS = 40              # Number of evaluated sample sizes
F1_THRESHOLD = 0.90        # Target F1 threshold
CONF_LEVEL = 0.95          # Confidence level
TEST_SIZE = 0.25           # Fixed test set size
USE_PRETRAINED=True              # Whether to train the models or just load
results from disk

# ==========================
# Data Loading & Preprocessing
# ==========================

def load_and_preprocess(csv_path: str):
    df = pd.read_csv(csv_path)
    df = df.dropna().reset_index(drop=True)

    if "individual_id" in df.columns:
        df["individual_id"] = df["individual_id"].astype("category").cat.codes

    y = df["species"]
    X = df.drop(columns=["species"])

    categorical_cols = X.select_dtypes(include=["object", "category"]).columns
    numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns

    return X, y, numeric_cols.tolist(), categorical_cols.tolist()
```

```python
# ========================
# Train / Test Split
# ========================

def split_data(X, y):
    return train_test_split(
        X,
        y,
        test_size=TEST_SIZE,
        stratify=y,
        random_state=RANDOM_STATE,
    )


# ========================
# Models
# ========================

def build_model(model_name, preprocessor):
    match model_name:
        case "Logistic Regression":
            return Pipeline([
                    ("prep", preprocessor),
                    ("clf", LogisticRegression(
                        max_iter=1000,
                        class_weight="balanced",
                        random_state=RANDOM_STATE
                    ))
                ])

        case "KNN":
            return Pipeline([
                    ("prep", preprocessor),
                    ("clf", KNeighborsClassifier(n_neighbors=5))
                ])

        case "Random Forest":
            return Pipeline([
                    ("prep", preprocessor),
                    ("clf", RandomForestClassifier(
                        n_estimators=50,
                        class_weight="balanced",
                        random_state=RANDOM_STATE
                    ))
                ])
```

```python
# ==========================
# Utilities
# ==========================

def generate_sample_sizes(y_train):
    n_classes = y_train.nunique()
    min_n = max(10, n_classes * 3) #minimum adequate sample size:
    # 3 samples per class, but at least 10 samples to start with

    sizes = np.linspace(
        min_n,
        len(y_train),
        N_POINTS,
        dtype=int
    )

    return sizes, min_n


def mean_confidence_interval(values):
    values = np.array(values)
    mean = values.mean()
    std = values.std(ddof=1)

    h = t.ppf(
        (1 + CONF_LEVEL) / 2,
        len(values) - 1
    ) * std / np.sqrt(len(values))

    return mean, mean - h, mean + h


# ==========================
# Sampling Strategies
# ==========================

def simple_random_sample(X, y, n):
    idx = np.random.choice(len(X), n, replace=False)
    return X.iloc[idx], y.iloc[idx]


def bernoulli_sample(X, y, p):
    mask = np.random.rand(len(X)) < p
```

```
    return X[mask], y[mask]


def systematic_sample(X, y, n):
    step = len(X) // n
    start = np.random.randint(0, step)
    idx = np.arange(start, start + step * n, step)
    return X.iloc[idx], y.iloc[idx]


def stratified_sample(X, y, n):

    strata = X["island"].astype(str) + "_" + y.astype(str)

    df_tmp = X.copy()
    df_tmp["y"] = y
    df_tmp["strata"] = strata

    sampled = (
        df_tmp
        .groupby("strata", group_keys=False)
        .apply(
            lambda g: g.sample(
                max(1, int(len(g) / len(df_tmp) * n))  # Proportional allocation
with at least 1 sample per stratum
            ),
            include_groups=False   # Fixes DeprecationWarning
        )
    )

    return sampled.drop(columns=["y"]), sampled["y"]


def build_sampler(sampling_name, X_train, y_train):
    match sampling_name:
        case "Simple Random":
            return lambda n: simple_random_sample(X_train, y_train, n)

        case "Bernoulli":
            return lambda n: bernoulli_sample(X_train, y_train, n / len(X_train))

        case "Systematic":
            return lambda n: systematic_sample(X_train, y_train, n)

        case "Stratified":
```

```python
            return lambda n: stratified_sample(X_train, y_train, n)

        case _:
            raise ValueError(f"Unknown sampling method: {sampling_name}")

# ==========================
# Core Experiment
# ==========================

def run_single_experiment(
    sampling_name, model_name, X_train, y_train, X_test, y_test,
    dataset_info_numeric_cols, dataset_info_categorical_cols,
    sample_sizes, min_n, n_repeats_per_sample
    ):

    sampler = build_sampler(sampling_name, X_train, y_train)

    preprocessor = ColumnTransformer(
        transformers=[
            ("num", StandardScaler(), dataset_info_numeric_cols),
            ("cat", OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ), dataset_info_categorical_cols),
        ],
        remainder="drop",
    )

    model = build_model(model_name, preprocessor)

    worker_results = {
        "sample_size": [],
        "f1_mean": [],
        "f1_ci_low": [],
        "f1_ci_high": []
    }

    for n in sample_sizes:
        f1_scores = []

        for _ in range(n_repeats_per_sample):
            X_s, y_s = sampler(n)
            if len(X_s) < min_n:
                continue
```

```
            model.fit(X_s, y_s)
            preds = model.predict(X_test)

            f1_scores.append(
                f1_score(y_test, preds, average="macro")
            )

        if len(f1_scores) < n_repeats_per_sample // 2: # skip if too few valid
samples were obtained.
            continue

        f1_mean, f1_low, f1_high = mean_confidence_interval(f1_scores)

        worker_results["sample_size"].append(n)
        worker_results["f1_mean"].append(f1_mean)
        worker_results["f1_ci_low"].append(f1_low)
        worker_results["f1_ci_high"].append(f1_high)

    return (sampling_name, model_name, worker_results)



def run_experiments(X_train, y_train, X_test, y_test,
                    dataset_info_numeric_cols, dataset_info_categorical_cols,
sample_sizes, min_n):

    sampling_names = ["Simple Random", "Bernoulli", "Systematic", "Stratified"]
    model_names = ["Logistic Regression", "KNN", "Random Forest"]

    results = {}

    tasks = [
        (s_name, m_name)
        for s_name in sampling_names
        for m_name in model_names
    ]

    raw_results = Parallel(n_jobs=-1)(
        delayed(run_single_experiment)(
            s_name, m_name,
            X_train, y_train, X_test, y_test,
            dataset_info_numeric_cols, dataset_info_categorical_cols,
            sample_sizes, min_n, N_REPEATS
        ) for s_name, m_name in tasks
    )
```

```
    results = {}
    for sampling_name, model_name, data in raw_results:
        if sampling_name not in results:
            results[sampling_name] = {}
        results[sampling_name][model_name] = data


    CI_table_dict = {}
    for  model_name in model_names:

        if model_name not in CI_table_dict:
                CI_table_dict[model_name] = {}

        for sampling_name in sampling_names:
            combination_data = results[sampling_name][model_name]

            samples_generated = combination_data["sample_size"]
            f1_means = combination_data["f1_mean"]
            f1_lows = combination_data["f1_ci_low"]
            f1_highs = combination_data["f1_ci_high"]

            #populate CI table for further visualizations

            for n, f1_mean, f1_low, f1_high in zip(samples_generated, f1_means,
f1_lows, f1_highs):
                status = "REACHED" if f1_mean >= F1_THRESHOLD else "NOT REACHED"
                value = f"{status}. F1 mean: {f1_mean*100:.1f}. F1 CI
[{f1_low*100:.1f}, {f1_high*100:.1f}]"

                CI_sampling_table_row_key = n
                CI_sampling_table_column_key = sampling_name

                if CI_sampling_table_row_key not in CI_table_dict[model_name]:
                    CI_table_dict[model_name][CI_sampling_table_row_key] = {}

                CI_table_dict[model_name][CI_sampling_table_row_key][CI_sampling_
table_column_key] = value


    CI_tables = {}
    for model_name in CI_table_dict.keys():
        CI_tables[model_name] = pd.DataFrame.from_dict(CI_table_dict[model_name],
orient='index')
```

```
    return results, CI_tables

# ========================
# Visualization
# ========================


def visualize_results(results):

    sampling_names = ["Simple Random", "Bernoulli", "Systematic", "Stratified"]
    model_names = ["Logistic Regression", "KNN", "Random Forest"]

    for sampling_name in sampling_names:

        plt.figure(figsize=(18, 5))

        for i, model_name in enumerate(model_names, 1):
            combination_data = results[sampling_name][model_name]

            samples_generated = combination_data["sample_size"]
            f1_means = combination_data["f1_mean"]
            f1_lows = combination_data["f1_ci_low"]
            f1_highs = combination_data["f1_ci_high"]

            # Custom x=axis ticks to show only a few key sample sizes and avoid
clutter
            #========================
            log_ticks = np.log2(samples_generated)
            # Determine how many labels we actually want to see
            illustrative_interval_size = 6
            nonillustrative_interval_size = 4
            num_points = len(samples_generated)

            # Generate the indices we want to "keep" and what will be the noise.
            if num_points <= illustrative_interval_size:
                show_indices = np.arange(num_points) # Show all if small enough
            else:
                show_indices = np.concatenate([
                    np.arange(illustrative_interval_size),
                    np.linspace(illustrative_interval_size*1.5, num_points - 1,
nonillustrative_interval_size, dtype=int)
                ])

            # Create the tick labels with empty strings for the "noise"
            tick_labels = [
```

```
                str(int(s)) if i in show_indices else ""
                for i, s in enumerate(samples_generated)
            ]
            #==========================

            # Plot
            ax = plt.subplot(1, 3, i)

            means_pct = np.array(f1_means) * 100
            lows_pct = np.array(f1_lows) * 100
            highs_pct = np.minimum(np.array(f1_highs), 1.0) * 100

            ax.plot(log_ticks, means_pct)
            ax.fill_between(log_ticks, lows_pct, highs_pct, alpha=0.3)

            ax.set_title(model_name)
            ax.set_xlabel("Sample size (log scale)")
            ax.set_ylabel("F1 score (%)")

            ax.set_xticks(log_ticks)
            ax.set_xticklabels(tick_labels, rotation=45)

            ax.set_ylim(50, 108)
            ax.axhline(F1_THRESHOLD * 100, color="red", linestyle="--", label="F1
Threshold: 90%")
            ax.axhline(100, color="green", linestyle="--", label="100%")
            ax.grid(True, which="both", linestyle="--", linewidth=0.5)

        plt.suptitle(f"{sampling_name} sampling", fontsize=16)
        plt.tight_layout()
        plt.show()

# ==========================
# Entry Point
# ==========================


if __name__ == "__main__":

    if USE_PRETRAINED and os.path.exists('experiment_results.pkl'):
        print("Loading pre-trained results from disk...")
        results = joblib.load('experiment_results.pkl')
        print("metedata:", results.get('metadata', {}))
        visualize_results(results)
```

```
    else:
        X, y, dataset_info_numeric_cols, dataset_info_categorical_cols =
load_and_preprocess("penguins.csv")
        X_train, X_test, y_train, y_test = split_data(X, y)
        print("len x_train:", len(X_train))
        print("len x_test:", len(X_test))

        sample_sizes, min_n = generate_sample_sizes(y_train)

        results, CI_tables = run_experiments(
            X_train, y_train,
            X_test, y_test,
            dataset_info_numeric_cols, dataset_info_categorical_cols,
            sample_sizes,
            min_n
        )

        results['metadata'] = {
        'threshold': F1_THRESHOLD,
        'timestamp': datetime.datetime.now(),
        'sample_sizes': sample_sizes
        }
        joblib.dump(results, 'experiment_results.pkl')

        visualize_results(results)

        for model_name, t in CI_tables.items():
            filename = f"f1_threshold_confidence_intervals__{model_name}.csv"
            t.to_csv(filename)

            print(f"\n=== {model_name}: Threshold confidence interval table saved
to {filename}")
```